

SINAU E-Learning

Panduan Alur Kode CRUD

Fitur Kelas & Materi - Framework Laravel (MVC)

BAB 1 - PENDAHULUAN

Panduan ini menjelaskan alur kode CRUD (Create, Read, Update, Delete) pada sistem e-learning SINAU yang dibangun menggunakan framework Laravel dengan pola arsitektur MVC (Model-View-Controller). Fitur yang digunakan sebagai contoh adalah Kelas dan Materi.

1.1 Arsitektur MVC Laravel

Laravel mengikuti pola MVC yang memisahkan logika aplikasi menjadi tiga lapisan:

- * Model - merepresentasikan data dan aturan bisnis (berinteraksi dengan database).
- * View - menampilkan data kepada pengguna (template Blade).
- * Controller - menerima permintaan dari pengguna, memproses melalui Model, lalu mengembalikan View.

1.2 Alur Request-Response

1 Browser / User

Pengguna mengisi form dan menekan tombol Submit di halaman View.

2 Route (web.php)

Laravel mencocokkan URL + HTTP Method ke method Controller yang sesuai.

3 Controller

Menerima Request, menjalankan validasi, memanggil Model untuk operasi DB.

4 Model (Eloquent ORM)

Menjalankan query SQL ke database (insert/select/update/delete).

5 Database (MySQL)

Menyimpan atau mengambil data sesuai perintah ORM.

6 Response / Redirect

Controller mengembalikan View atau redirect dengan pesan sukses/gagal.

BAB 2 - STRUKTUR FILE PROYEK

Berikut adalah lokasi file-file yang berkaitan dengan fitur Kelas dan Materi:

```
app/  
  Http/  
    Controllers/  
      KelasController.php    <- Controller utama fitur Kelas  
      MateriController.php   <- Controller utama fitur Materi  
    Models/  
      Kelas.php              <- Model Eloquent tabel 'kelas'  
      Materi.php             <- Model Eloquent tabel 'materi'  
  
  routes/  
    web.php                  <- Definisi seluruh route aplikasi  
  
database/  
  migrations/  
    2026_09_20_145305_create_kelas_tabel.php  
    2026_09_20_145727_create_materi_tabel.php  
  
resources/views/  
  guru/  
    kelas/  
      index.blade.php        <- Daftar kelas (termasuk tombol aksi)  
      create.blade.php       <- Modal form tambah kelas  
      edit.blade.php         <- Modal form edit kelas  
      detail.blade.php       <- Modal detail kelas  
    materi/  
      index.blade.php        <- Daftar materi + modal tambah & edit  
      create.blade.php       <- Form body modal tambah materi
```

BAB 3 - ROUTE (routes/web.php)

File routes/web.php adalah pintu masuk semua permintaan HTTP. Setiap baris mendefinisikan: HTTP Method, URL Pattern, Controller, dan Method yang dipanggil.

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\KelasController;
use App\Http\Controllers\MateriController;

// --- ROUTE KELAS ---
// GET /guru => KelasController@index
// Menampilkan dashboard guru + semua kelas, kuis.
Route::get('/guru', [KelasController::class, 'index'])->name('guru');

// POST /guru/kelas => KelasController@store
// Menyimpan kelas baru dari form tambah.
Route::post('/guru/kelas', [KelasController::class, 'store'])->name('guru.kelas.store');

// GET /guru/kelas/{id} => KelasController@show
// Data detail satu kelas (JSON untuk modal detail).
Route::get('/guru/kelas/{id}', [KelasController::class, 'show'])->name('guru.kelas.show');

// PUT /guru/kelas/{id} => KelasController@update
// Memperbarui data kelas berdasarkan ID.
Route::put('/guru/kelas/{id}', [KelasController::class, 'update'])->name('guru.kelas.update');

// DELETE /guru/kelas/{id} => KelasController@destroy
// Menghapus kelas berdasarkan ID.
Route::delete('/guru/kelas/{id}', [KelasController::class, 'destroy'])->name('guru.kelas.destroy');

// --- ROUTE MATERI ---
Route::post('/guru/materi', [MateriController::class, 'store'])->name('guru.materi.store');
Route::put('/guru/materi/{id}', [MateriController::class, 'update'])->name('guru.materi.update');
Route::delete('/guru/materi/{id}', [MateriController::class, 'destroy'])->name('guru.materi.destroy');
Route::get('/mater/download/{id}', [MateriController::class, 'download'])->name('mater.download');
```

[i] Catatan

Nama route (->name(...)) memungkinkan penggunaan helper route('nama') di View dan Controller sehingga URL tidak perlu di-hardcode. Jika URL berubah, cukup ubah di satu tempat di web.php.

BAB 4 - MIGRATION (Struktur Tabel Database)

4.1 Tabel kelas

Migration mendefinisikan struktur tabel di database. Dijalankan dengan: `php artisan migrate`

```
Schema::create('kelas', function (Blueprint $table) {
    $table->id(); // Primary Key (auto-increment)
    $table->foreignId('guru_id') // Foreign key ke tabel users
        ->constrained('users') // referensi kolom id di tabel users
        ->onDelete('cascade'); // jika user dihapus, kelas ikut terhapus
    $table->string('nama_kelas'); // Nama kelas, max 255 karakter
    $table->string('mata_pelajaran'); // Mata pelajaran yang diajarkan
    $table->string('kode_kelas')->unique(); // Kode unik (contoh: FIS10B)
    $table->text('deskripsi')->nullable(); // Deskripsi/jadwal (boleh kosong)
    $table->timestamps(); // created_at & updated_at (otomatis Laravel)
});
```

4.2 Tabel materi

```
Schema::create('materi', function (Blueprint $table) {
    $table->id(); // Primary Key
    $table->foreignId('kelas_id') // Foreign key ke tabel kelas
        ->constrained('kelas') // referensi ke tabel kelas
        ->onDelete('cascade'); // jika kelas dihapus, materi ikut terhapus
    $table->string('judul'); // Judul materi/modul
    $table->text('deskripsi')->nullable(); // Deskripsi singkat
    $table->string('file_path')->nullable(); // Path file di storage
    $table->integer('urutan')->default(1); // Urutan tampil materi
    $table->timestamps();
});
```

[i] Catatan

`onDelete('cascade')` adalah fitur penting: jika baris induk (kelas) dihapus, semua baris anak (materi yang terhubung) akan otomatis ikut terhapus oleh database. Ini mencegah data orphan (materi yang mengacu ke kelas yang tidak ada).

BAB 5 - MODEL ELOQUENT

Model Eloquent adalah representasi OOP dari tabel database. Setiap baris tabel = satu objek Model. Model juga mendefinisikan relasi antar tabel.

5.1 Model Kelas (app/Models/Kelas.php)

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Kelas extends Model
{
    // Nama tabel di database
    protected $table = 'kelas';

    // Mass Assignment: kolom yang boleh diisi via create() atau fill()
    // Kolom di luar daftar ini TIDAK BISA diisi secara massal (keamanan)
    protected $fillable = [
        'guru_id',
        'nama_kelas',
        'mata_pelajaran',
        'kode_kelas',
        'deskripsi',
    ];

    // RELASI: Setiap kelas dimiliki oleh satu Guru (User)
    // belongsTo = 'many-to-one' (banyak kelas -> satu guru)
    public function guru()
    {
        return $this->belongsTo(User::class, 'guru_id');
    }

    // RELASI: Satu kelas memiliki banyak Materi
    // hasMany = 'one-to-many' (satu kelas -> banyak materi)
    public function materi()
    {
        return $this->hasMany(Materi::class, 'kelas_id');
    }

    // RELASI: Satu kelas memiliki banyak Kuis
    public function kuis()
    {
        return $this->hasMany(Kuis::class, 'kelas_id');
    }
}
```

[i] Keuntungan Relasi Eloquent

Relasi Eloquent memudahkan pengambilan data terkait tanpa menulis JOIN secara manual. Contoh: `$kelas->materi->count()` menghasilkan: `SELECT COUNT(*) FROM materi WHERE kelas_id = ?`

5.2 Model Materi (app/Models/Materi.php)

BAB 6 - CONTROLLER KELAS (KelasController.php)

Controller adalah otak aplikasi. Menerima data dari request, memvalidasi, memanggil Model, dan mengembalikan response ke pengguna.

6.1 Method index() - Menampilkan Dashboard Guru (READ ALL)

Method ini dipanggil saat guru membuka halaman /guru. Fungsinya: memuat semua data kelas, materi, kuis lalu dikirim ke View.

```
public function index(Request $request)
{
    // Ambil parameter pencarian dari URL (?q=kata_kunci)
    $search = $request->input('q');

    // Buat query Eloquent dengan eager loading relasi guru, materi, kuis
    // Eager loading mencegah 'N+1 Query Problem'
    $kelasQuery = Kelas::with(['guru', 'materi', 'kuis']);

    // Jika ada kata pencarian, tambahkan filter WHERE
    if ($search) {
        $kelasQuery->where(function ($query) use ($search) {
            $query->where('nama_kelas', 'like', "%{$search}%")
                ->orWhere('mata_pelajaran', 'like', "%{$search}%")
                ->orWhere('kode_kelas', 'like', "%{$search}%");
        });
    }

    // Eksekusi query: ambil semua data terbaru (ORDER BY created_at DESC)
    $kelasList = $kelasQuery->latest()->get();
    $materiList = Materi::with('kelas')->latest()->get();
    $kuisList = Kuis::with(['kelas', 'soal'])->latest()->get();

    // Kirim data ke View guru.dashboard
    // compact() = shortcut ['kelasList'=>$kelasList, 'materiList'=>$materiList, ...]
    return view('guru.dashboard', compact('kelasList', 'materiList', 'kuisList', 'search'));
}
```

6.2 Method store() - Menyimpan Kelas Baru (CREATE)

```
public function store(Request $request)
{
    // LANGKAH 1: Validasi data form
    // Jika validasi gagal, Laravel otomatis redirect balik + mengirim pesan error
    $validated = $request->validate([
        'nama_kelas' => 'required|string|min:3|max:100',
        'mata_pelajaran' => 'required|string|max:100',
        'deskripsi' => 'nullable|string|max:255',
    ], [
        'nama_kelas.required' => 'Nama kelas wajib diisi.',
        'nama_kelas.min' => 'Nama kelas minimal harus 3 karakter.',
    ]);
}
```


6.3 Method show() - Detail Kelas (READ by ID)

```
public function show($id)
{
    // findOrFail(): ambil kelas berdasarkan ID
    // Jika tidak ditemukan, otomatis throw 404 Not Found
    // with(...): eager load relasi sekaligus
    $kelas = Kelas::with(['guru', 'materi', 'kuis.soal'])->findOrFail($id);

    // Jika request dari JavaScript (AJAX/fetch), kirim JSON
    if (request()->wantsJson() || request()->ajax()) {
        return response()->json($kelas);
    }

    return redirect('/guru#kelas');
}
```

6.4 Method update() - Memperbarui Kelas (UPDATE)

```
public function update(Request $request, $id)
{
    // LANGKAH 1: Cari data kelas yang akan diupdate
    $kelas = Kelas::findOrFail($id); // Lempar 404 jika tidak ada

    // LANGKAH 2: Validasi data baru dari form
    $validated = $request->validate([
        'nama_kelas'      => 'required|string|min:3|max:100',
        'mata_pelajaran' => 'required|string|max:100',
        'deskripsi'       => 'nullable|string|max:255',
    ]);

    // LANGKAH 3: Update kolom-kolom yang ada di $fillable
    $kelas->update($validated);

    // LANGKAH 4: Redirect dengan pesan sukses
    return redirect('/guru#kelas')
        ->with('success', "Kelas '{$kelas->nama_kelas}' berhasil diperbarui!");
}
```

6.5 Method destroy() - Menghapus Kelas (DELETE)

```
public function destroy($id)
{
    // LANGKAH 1: Cari kelas yang akan dihapus
    $kelas = Kelas::findOrFail($id);
    $nama = $kelas->nama_kelas; // Simpan nama sebelum dihapus

    // LANGKAH 2: Hapus dari database
    // Karena relasi onDelete('cascade'), semua materi & kuis kelas ini
    // juga akan otomatis ikut terhapus oleh database
}
```


BAB 7 - CONTROLLER MATERI (MateriController.php)

MateriController mengelola upload file modul pembelajaran. Fitur unggah file menggunakan Laravel Storage untuk menyimpan file secara aman.

7.1 Method store() - Unggah Materi Baru (CREATE)

```
public function store(Request $request)
{
    // LANGKAH 1: Validasi, termasuk validasi file upload
    $validated = $request->validate([
        'kelas_id' => 'required|exists:kelas,id', // ID kelas harus ada di DB
        'judul'    => 'required|string|min:3|max:255',
        'deskripsi' => 'nullable|string|max:1000',
        'file_materi' => 'nullable|file|mimes:pdf,doc,docx,ppt,pptx,zip|max:20480',
        // nullable = file boleh tidak diunggah
        // mimes     = hanya ekstensi ini yang diizinkan
        // max:20480 = maksimal 20 MB (20480 KB)
    ]);

    // LANGKAH 2: Proses upload file (jika ada)
    $filePath = null;
    if ($request->hasFile('file_materi')) {
        // store() menyimpan file ke storage/app/public/materi/
        // dan mengembalikan path relatif, contoh: 'materi/abc123.pdf'
        $filePath = $request->file('file_materi')->store('materi', 'public');
    }

    // LANGKAH 3: Simpan record materi ke database
    Materi::create([
        'kelas_id' => $validated['kelas_id'],
        'judul'    => $validated['judul'],
        'deskripsi' => $validated['deskripsi'] ?? null,
        'file_path' => $filePath, // null jika tidak ada file
    ]);

    return redirect('/guru#materi')->with('success', 'Materi baru berhasil diunggah!');
}
```

7.2 Method update() - Perbarui Materi (UPDATE)

```
public function update(Request $request, $id)
{
    $materi = Materi::findOrFail($id);

    // Validasi sama seperti store()
    $validated = $request->validate([...]);

    // Jika ada file BARU diunggah:
    if ($request->hasFile('file_materi')) {
        // HAPUS file lama dari storage agar tidak menumpuk
        if ($materi->file_path && Storage::disk('public')->exists($materi->file_path)) {
```

```

        Storage::disk('public')->delete($materi->file_path);
    }
    // Simpan file baru, update path di model
    $materi->file_path = $request->file('file_materi')->store('materi', 'public');
}

// Update kolom teks
$materi->kelas_id = $validated['kelas_id'];
$materi->judul = $validated['judul'];
$materi->deskripsi = $validated['deskripsi'] ?? null;
$materi->save(); // save() = UPDATE SQL query ke database

return redirect('/guru#materi')->with('success', '...');
}

```

7.3 Method destroy() - Hapus Materi + File (DELETE)

```

public function destroy($id)
{
    $materi = Materi::findOrFail($id);

    // Penting: hapus file fisik dari storage sebelum hapus record DB
    // Jika tidak, file akan tetap ada di server (storage leak)
    if ($materi->file_path && Storage::disk('public')->exists($materi->file_path)) {
        Storage::disk('public')->delete($materi->file_path);
    }

    $materi->delete(); // Hapus record dari tabel materi

    return redirect('/guru#materi')->with('success', 'Materi berhasil dihapus!');
}

```

7.4 Method download() - Unduh File Materi

```

public function download($id)
{
    $materi = Materi::findOrFail($id);

    // Validasi: pastikan file ada di storage
    if (!$materi->file_path || !Storage::disk('public')->exists($materi->file_path)) {
        return redirect()->back()->with('error', 'Berkas tidak ditemukan di server.');
```

```

    }

    // Kembalikan file sebagai response unduhan browser
    return Storage::disk('public')->download($materi->file_path);
}

```

BAB 8 - VIEW (Tampilan Blade)

View adalah template HTML yang menerima data dari Controller dan menampilkannya. Laravel menggunakan Blade sebagai template engine dengan sintaks @if, @foreach, {{ }}, dll.

8.1 Halaman Daftar Kelas (guru/kelas/index.blade.php)

```
{{-- $kelasList dikirim dari KelasController@index --}}
@if(isset($kelasList) && $kelasList->count() > 0)
    @foreach($kelasList as $kelas)
        <div class='kelas-item-card'
            data-nama='{{ strtolower($kelas->nama_kelas) }}'
            data-mapel='{{ strtolower($kelas->mata_pelajaran) }}'>

            {{-- Tampilkan nama, kode, mata pelajaran --}}
            <h3>{{ $kelas->nama_kelas }}</h3>
            <span>{{ $kelas->kode_kelas }}</span>

            {{-- Hitung materi & kuis via relasi Eloquent --}}
            {{ $kelas->materi->count() }} materi
            {{ $kelas->kuis->count() }} kuis

            {{-- TOMBOL DETAIL: kirim data ke JS sebagai JSON --}}
            <button onclick='bukaModalDetailKelas(@json($kelas))'>Detail</button>

            {{-- TOMBOL EDIT: kirim data ke JS untuk isi form --}}
            <button onclick='bukaModalEditKelas(@json($kelas))'>Edit</button>

            {{-- FORM HAPUS: menggunakan method spoofing DELETE --}}
            <form action='{{ route("guru.kelas.destroy", $kelas->id) }}' method='POST'
                onsubmit='return confirm("Hapus kelas {{ $kelas->nama_kelas }}?")'>
                @csrf {{-- Token keamanan CSRF --}}
                @method('DELETE') {{-- Spoofing: HTML form tidak dukung DELETE --}}
                <button type='submit'>Hapus</button>
            </form>
        </div>
    @endforeach
@else
    <p>Belum ada kelas. Buat kelas pertama Anda sekarang.</p>
@endif
```

[i] Mengapa @csrf Wajib?

@csrf menghasilkan hidden input berisi token keamanan. Laravel secara otomatis memverifikasi token ini di setiap request POST/PUT/DELETE. Tanpa ini, request akan ditolak dengan error 419 (CSRF Token Mismatch).

8.2 Form Tambah Kelas - Modal (guru/kelas/create.blade.php)

```
{{-- Modal overlay, tersembunyi by default --}}
<div id='modal-tambah-kelas' class='fixed inset-0 z-50 hidden'>

    <form action='{{ route("guru.kelas.store") }}' method='POST'>
```


BAB 9 - RINGKASAN ALUR CRUD

9.1 Alur CREATE (Tambah Kelas Baru)

1 User klik tombol '+ Tambah Kelas' di View

JavaScript membuka modal form. Form belum dikirim ke server.

2 User isi form & klik 'Buat Kelas'

Browser mengirim HTTP POST ke /guru/kelas. Body berisi nama_kelas, mata_pelajaran, deskripsi.

3 Route mencocokkan request

Route::post('/guru/kelas', [KelasController::class, 'store'])

4 KelasController@store() dijalankan

1) Validasi. 2) Generate kode unik. 3) Kelas::create([...]). 4) Redirect balik.

5 Browser menerima redirect

Halaman dimuat ulang. Flash message 'Kelas berhasil dibuat!' ditampilkan.

9.2 Alur UPDATE (Edit Kelas)

1 User klik tombol 'Edit'

JavaScript memanggil bukaModalEditKelas(@json(\$kelas)). Data kelas diisi ke form edit.

2 User ubah data & klik 'Simpan Perubahan'

Browser kirim HTTP POST ke /guru/kelas/{id} dengan hidden field _method=PUT.

3 Route mencocokkan

Route::put('/guru/kelas/{id}', [..., 'update'])

4 KelasController@update() dijalankan

1) Kelas::findOrFail(\$id). 2) Validasi. 3) \$kelas->update(\$validated). 4) Redirect.

9.3 Alur DELETE (Hapus Kelas)

1 User klik tombol 'Hapus'

Muncul dialog konfirmasi JS: 'Hapus kelas Fisika 10B?'

2 User klik 'OK'

Form dikirim ke /guru/kelas/{id} dengan _method=DELETE.

3 Route mencocokkan

Route::delete('/guru/kelas/{id}', [..., 'destroy'])

4 KelasController@destroy() dijalankan

1)findOrFail(\$id). 2) \$kelas->delete(). 3) DB cascade hapus materi terkait. 4) Redirect.

9.4 Alur UPLOAD FILE Materi

1 Guru buka modal 'Unggah Materi' & pilih file

Form menggunakan enctype='multipart/form-data' agar file bisa dikirim.

2 Submit form ke POST /guru/materi

MateriController@store() dipanggil.

3 Validasi & simpan file

```
$request->file()->store('materi', 'public') => storage/app/public/materi/
```

4 Simpan path ke database

```
Materi::create(['file_path' => $filePath])
```

5 Unduh file

```
GET /materi/download/{id} => Storage::disk('public')->download($materi->file_path)
```

BAB 10 - KONSEP PENTING LARAVEL

10.1 Validasi Request

```
// Rule validasi yang tersedia:
'field' => 'required'           // Wajib diisi
'field' => 'string'             // Harus berupa teks
'field' => 'min:3'              // Minimal 3 karakter
'field' => 'max:100'            // Maksimal 100 karakter
'field' => 'nullable'           // Boleh kosong/null
'field' => 'exists:kelas,id'     // Harus ada di tabel kelas kolom id
'field' => 'file'               // Harus berupa file upload
'field' => 'mimes:pdf,doc'       // Ekstensi file yang diizinkan
'field' => 'max:20480'           // Ukuran file maks 20MB

// Jika validasi GAGAL:
// - Otomatis redirect balik ke halaman sebelumnya
// - Error tersedia di $errors di View via @error('field')
// - Nilai lama tersedia via old('field_name')
```

10.2 Flash Message (Pesan Sementara)

```
// Di Controller, kirim flash message:
return redirect('/guru#kelas')->with('success', 'Berhasil!');

// Di View (resources/views/partials/alert.blade.php):
@if(session('success'))
    <div class='alert-success'>{{ session('success') }}</div>
@endif
@if(session('error'))
    <div class='alert-error'>{{ session('error') }}</div>
@endif
```

10.3 Eager Loading vs Lazy Loading

```
// TANPA eager loading (Lazy Loading) - N+1 Problem!
// Jika ada 20 kelas, ini menjalankan 1 + 20 = 21 query!
$kelasList = Kelas::all();
foreach ($kelasList as $kelas) {
    echo $kelas->materi->count(); // query baru per kelas!
}

// DENGAN eager loading - hanya 2 query total!
// Query 1: SELECT * FROM kelas
// Query 2: SELECT * FROM materi WHERE kelas_id IN (1,2,3,...)
$kelasList = Kelas::with(['materi', 'kuis'])->get();
foreach ($kelasList as $kelas) {
    echo $kelas->materi->count(); // data sudah di-cache!
}
```

10.4 Method Spoofing (PUT & DELETE dari Form HTML)

```
// HTML form hanya mendukung GET dan POST.
// Untuk PUT/DELETE, Laravel membaca hidden field _method:
<form action='/guru/kelas/5' method='POST'>
    @csrf
    @method('DELETE')
    {{-- Menghasilkan: <input type='hidden' name='_method' value='DELETE'> --}}
    <button>Hapus</button>
</form>

// Laravel membaca _method dan memperlakukannya sebagai DELETE,
// sehingga Route::delete() bisa cocok.
```

10.5 Storage - Penyimpanan File

```
// File disimpan di: storage/app/public/materi/nama_file.pdf
// Bisa diakses publik via: php artisan storage:link
// URL akses: http://localhost:8000/storage/materi/nama_file.pdf

// Menyimpan file:
$path = $request->file('file_materi')->store('mater_i', 'public');
// $path = 'mater_i/7f3a9b2c.pdf' (path relatif yang disimpan di DB)

// Cek apakah file ada:
Storage::disk('public')->exists($path); // true/false

// Hapus file:
Storage::disk('public')->delete($path);

// Download file ke browser:
Storage::disk('public')->download($path);
```

GLOSARIUM ISTILAH

MVC	Model-View-Controller. Pola arsitektur pemisahan kode agar mudah dikelola.
Eloquent ORM	Object-Relational Mapping Laravel. Mengizinkan operasi DB menggunakan kode PHP OOP.
Migration	File PHP yang mendefinisikan struktur tabel database secara programatik.
Blade	Template engine Laravel. Memperluas HTML dengan sintaks @if, @foreach, {{ }}, dll.
Route	Aturan pemetaan URL + HTTP Method ke Controller method tertentu.
Request	Objek yang merepresentasikan permintaan HTTP (URL, method, body, header).
Redirect	Mengarahkan browser ke URL lain setelah suatu aksi selesai.
Flash Message	Pesan sementara yang disimpan di session dan ditampilkan sekali saja.
CSRF	Cross-Site Request Forgery. Token keamanan agar form tidak bisa dipalsukan.
Eager Loading	Mengambil data relasi sekaligus dalam satu query untuk efisiensi.
\$fillable	Daftar kolom model yang boleh diisi menggunakan create() atau fill().
findOrFail()	Mencari data by ID. Lempar 404 otomatis jika data tidak ditemukan.
compact()	Helper PHP untuk membuat array asosiatif dari nama variabel.
Storage	Facade Laravel untuk mengelola file di filesystem (simpan, hapus, unduh).
Method Spoofing	Teknik mengirim PUT/DELETE melalui form HTML biasa menggunakan _method.
belongsTo	Relasi Eloquent: model ini memiliki foreign key yang mengacu ke model lain.
hasMany	Relasi Eloquent: satu record model ini berhubungan dengan banyak record lain.
onDelete cascade	Aturan DB: jika baris induk dihapus, baris anak ikut dihapus otomatis.

